

Homework 2 Part I: GPT Decoder-Only Transformer for NLP

Anatolie Jentimir
COMP.5300: Deep Learning, LLMs, and LVMs

May 6, 2026

1 Task and Dataset

For Part I, I implemented an unconditional language model using a GPT-style decoder-only Transformer. The model is trained with a causal next-token prediction objective. For a token sequence

$$x_0, x_1, x_2, \dots, x_T,$$

the model receives

$$x_{\text{in}} = x_0, x_1, \dots, x_{T-1}$$

and predicts

$$y_{\text{target}} = x_1, x_2, \dots, x_T.$$

I used the Tiny Shakespeare dataset because it is small enough for homework-scale training while still being useful for testing autoregressive text generation. The dataset source is:

<https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>

The model uses character-level tokenization. I used a 90% training and 10% validation split. The default context length is 128 tokens. I reserved two special tokens: PAD=0 and EOS=1. Padding is not central for fixed-length character chunks, but it is included for correct masking behavior and future extension.

2 Model Architecture

The model is a decoder-only Transformer implemented from scratch with basic PyTorch layers. It does not use the following forbidden prebuilt Transformer components: `torch.nn.Transformer`, `torch.nn.TransformerDecoder`, `torch.nn.MultiheadAttention`, or pretrained GPT blocks.

The architecture contains:

- token embeddings, shape $(B, T) \rightarrow (B, T, d_{\text{model}})$;
- learnable positional embeddings;
- a stack of GPT decoder blocks;
- masked multi-head self-attention;
- feed-forward network with GELU activation;
- residual connections and Pre-LayerNorm;
- final LayerNorm;
- language modeling head, shape $(B, T, d_{\text{model}}) \rightarrow (B, T, V)$.

The default configuration used in my code is shown in Table 1.

Hyperparameter	Value
Context length	128
Embedding size d_{model}	128
Number of heads	4
Number of decoder blocks	2
Feed-forward size d_{ff}	512
Dropout	0.1
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	0.01
Seed	42

Table 1: Default model and training configuration.

3 Causal Self-Attention and Masking

The key requirement in a GPT-style decoder-only model is that each token can only attend to previous tokens and itself. Future positions must be blocked. The causal mask has shape

$$(1, 1, T, T).$$

It is created as an upper-triangular Boolean mask where `True` means *block this position*. The attention scores have shape

$$(B, H, T, T),$$

where B is batch size, H is the number of heads, and T is sequence length.

The attention computation follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_h}} + M \right) V,$$

where masked positions are replaced by a large negative value before softmax.

Listing 1: Causal and padding mask logic.

```
def make_causal_mask(T, device):
    return torch.triu(
        torch.ones(T, T, dtype=torch.bool, device=device),
        diagonal=1
    ).unsqueeze(0).unsqueeze(0)

def make_padding_mask(token_ids, pad_id=0):
    return (token_ids == pad_id).unsqueeze(1).unsqueeze(2)

causal_mask = make_causal_mask(T, token_ids.device)
padding_mask = make_padding_mask(token_ids, pad_id)
combined_mask = causal_mask | padding_mask
scores = scores.masked_fill(combined_mask, -1e9)
```

Padding is handled twice: PAD keys are blocked in attention, and PAD targets are ignored in the cross-entropy loss using `ignore_index=PAD_ID`.

4 Training Objective

The model is trained with shifted next-token cross-entropy loss. The training code uses the following pattern:

Listing 2: Shifted next-token objective.

```
x_in = batch[:, :-1]
y_tgt = batch[:, 1:]

logits, _ = model(x_in)
loss = F.cross_entropy(
    logits.reshape(-1, vocab_size),
    y_tgt.reshape(-1),
    ignore_index=PAD_ID
)
```

The optimizer is AdamW with weight decay. I also use gradient clipping and a warmup plus cosine decay learning rate schedule. During training, the script saves the best validation checkpoint, the tokenizer, the configuration file, a CSV training log, a loss plot, and final validation results.

5 Evaluation and Generation

The model is evaluated with validation negative log-likelihood (NLL) and perplexity:

$$\text{PPL} = e^{\text{mean NLL}}.$$

After running training, the exact values should be copied from:

- outputs/final_results.json
- outputs/training_log.csv

Metric	Value
Best validation NLL	Fill from outputs/final_results.json
Best validation PPL	Fill from outputs/final_results.json
Final training loss	Fill from outputs/training_log.csv

Table 2: Validation results. Fill values after running train.py.

I tested two decoding methods: greedy decoding and sampling. Greedy decoding always selects the highest-probability next token, so it is deterministic but can become repetitive. Sampling draws from the output distribution and can use temperature, top- k , or top- p filtering, so it usually produces more diverse text.

Example generation commands:

```
python generate.py --prompt "ROMEO:" --method greedy --max_new_tokens 300
python generate.py --prompt "ROMEO:" --method sample --temperature 0.8 --top_k
    20 --max_new_tokens 300
python generate.py --prompt "JULIET:" --method sample --temperature 0.9 --
    top_p 0.9 --max_new_tokens 300
```

For the final submission, I will include at least 10 generated samples with prompts and decoding settings. The recommended prompts are ROMEO:, JULIET:, KING:, To be, and My lord.

6 Analysis

6.1 Greedy Decoding vs. Sampling

Greedy decoding is stable and reproducible, but it may repeat common patterns because it always chooses the most likely next character. Sampling is less deterministic and produces more varied text. With a moderate temperature such as 0.8 and $\text{top-}k = 20$, the model can produce more diverse Shakespeare-like text while still avoiding very unlikely characters.

6.2 Common Failure Modes

Because this is a small character-level model, it may learn local spelling, punctuation, and line-break patterns before learning long-range structure. Common failure modes include repeated words, broken names, inconsistent dialogue structure, and text that looks Shakespeare-like at the character level but does not always have coherent meaning. These limitations are expected for a small model, short context length, and limited training time.

7 Reproducibility

The main reproducible training command is:

```
python train.py --seed 42 --epochs 5 --batch_size 16 --block_size 128 --  
d_model 128 --n_heads 4 --n_layers 2 --lr 3e-4
```

The code saves all important artifacts in the `outputs/` folder. These include the model checkpoint, tokenizer, training log, configuration file, final result JSON, and loss plot. This allows the training setup and reported results to be checked later.

8 Conclusion

This project implements the core mechanism of GPT-style language modeling: a decoder-only Transformer with causal self-attention and a shifted next-token prediction objective. The causal mask ensures that each token only uses past context, while the language modeling head predicts the next token at every position. The implementation is small but complete enough to demonstrate training, validation perplexity, and text generation using greedy and sampling-based decoding.

References

- [1] A. Karpathy, *Tiny Shakespeare dataset*, char-rnn GitHub repository. Available: <https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>
- [2] A. Vaswani et al., “Attention Is All You Need,” 2017.
- [3] PyTorch Documentation. Available: <https://pytorch.org/docs/stable/index.html>